FREE PROMPT PACK · CLAUDE CODE · REA

Give Your Jarvis *X-ray* Vision

Twelve copy-paste prompts in two parts. Part One builds the Jarvis from my videos: a galaxy of your own notes, a brain that answers from them, a voice both ways, a butler, a memory that grows when you talk, and a preflight that keeps it honest. Part Two gives him X-ray vision with REA, the open-source reverse engineering kit. Say "how does this site do that?" and a clean browser opens, REA watches while Jarvis scrolls, and about forty-five seconds later you hear the answer and get a teardown with the real stack, every file the site loads, and a recipe to rebuild it.

ZUBAIR TRABZADA

AI WORKSHOP

OCTOBER 2026

How to use this pack

What you need: Claude Code (I build with Claude Opus 5.5; type /model to pick it), Google Chrome, Node.js 22.19 or newer (free from nodejs.org), a folder of markdown notes, and an Anthropic API key for the brain. REA itself is free, open source under the MIT licence, and needs no key.

Set up in one line. Open a terminal, make an empty folder, and start Claude Code inside it: `mkdir my-jarvis && cd my-jarvis && claude`. Then paste Prompt 01.

1 **Part Two assumes Part One.**
The teardown prompts bolt onto the server, viewer and memory that Part One creates. If you already built Jarvis from an earlier pack, skip straight to Prompt 07; the prompts refer to "the server" and "the viewer" and Claude Code will find them.

2 **Paste each prompt whole, unedited, the first time.**
Every prompt is a complete brief: what to build, the laws it must respect, and how to prove it works. The verification at the end of each one is where the reliability comes from.

3 **Never paste an API key into a chat window.**
Prompt 01 creates a placeholder in a config file. You type the real key into that file yourself, in your own editor. Treat anything pasted into any chat as exposed.

4 **Websites only, and only ones you may look at.**
Public websites, open-source code, and your own projects. REA reads what a site already sends to your browser, the same view any web developer has. It never logs in, clicks, or changes anything, and this pack keeps it that way.

5 **Test quietly, then clean up.**
Test tabs open with ?mute=1, test teardowns run with a hidden browser, and every test removes the notes it wrote. Prompt 12 builds that drill for you.

What REA changes, honestly

Ask any AI how a beautiful website was built and it will happily answer. The question is whether it is remembering, guessing, or looking. REA is what lets Jarvis look.

	JARVIS WITHOUT REA	JARVIS WITH REA
How he answers	From memory, a web search, or a screenshot.	**He inspects the live site**: its code, its files, and how it behaves while it scrolls.
What you get	A plausible guess: "probably Three.js and GSAP".	**The real stack with versions**, every file it loads, and how the effect actually works.
Proof	None.	**Evidence for every claim**, guesses labelled as guesses, and a list of what he could not see.
Afterwards	Gone when the conversation moves on.	**A teardown note in your second brain**, with a snapshot and a recipe to rebuild it.
What it cannot do	Know what it hasn't seen.	**See the server.** Anything that happens on the site's own servers stays invisible, and so does logic the browser never receives.

> **What is not REA.** The clean browser, the scroll, the asset log, the version fingerprints, the panel and the note are mechanics you build in Part Two. REA supplies the evidence. And REA is not magic: a skilled developer could find the same things by hand in their browser's developer tools. REA makes it automatic, repeatable, and backed by proof.

Part One · the foundation

Six prompts that build the Jarvis from the videos: a 3D galaxy of your own notes, a brain that answers only from them, a voice in both directions, the camera dive to the source note, a personality, a memory you grow by talking, and a preflight harness that tells you the truth about what works.

If you built Jarvis from an earlier pack, you already have all of this: skip to Part Two. Prompt 05, Total Recall, matters most here, because every teardown is born in the galaxy the same way a "remember that" note is.

PROMPT 01 · THE FOUNDATION

The Galaxy and the Brain

Best for: the first paste. One prompt builds the whole foundation and starts it.

```
Build me a 3D knowledge galaxy from my markdown notes, and give it a brain I can talk to. Do the whole thing in one pass, then start it and verify it before you tell me it's done.

MY NOTES: [PASTE THE FULL PATH TO YOUR NOTES FOLDER, or delete this line and write "I have no notes, so generate 30 realistic sample notes about a small business into ./notes first"]

THE INDEXER: build.py, Python 3, standard library only. Scan every .md file and write viewer/graph-data.js containing const GRAPH = {nodes: [...], links: [...]}. Each node: a label from the filename, its parent folder as a group, roughly a 700-character excerpt. Link two notes when one mentions the other's title or they share [[wikilinks]]. CRITICAL: every node gets a numeric id equal to its index in the nodes array. Later prompts look nodes up by index.

THE VIEWER: viewer/index.html, single page, 3d-force-graph from a CDN pinned to an exact version. No npm, no build step, no framework. Cinematic: black space, a starfield, nodes glowing and colour-coded by group, slow idle rotation. Clicking a node flies the camera to it, lights its neighbours, and opens a side panel with the excerpt.

THE SERVER: server.py, port 4700, bound to 127.0.0.1, serving ONLY the viewer/ folder. Create config.json in the PROJECT ROOT, never inside viewer/, containing {"anthropic_api_key": "PUT-YOUR-KEY-HERE", "model": "claude-opus-5-5"}. Do not ask me for the key and never put it anywhere the browser can reach.

THE BRAIN: add POST /chat. Score every note against my question by keyword overlap, weighting title matches higher, take the top 6, and call the Anthropic Messages API with the official anthropic Python package and the model from config.json. System prompt: answer ONLY from these notes, in two or three sentences, and say so plainly when the notes do not cover it. Return {"answer": "...", "nodes": [array indexes used]}. Keep a short conversation history server-side so follow-ups work. Add a clean input bar at the bottom of the viewer that shows the answer.

OPUS 5.5 RULES: thinking is always on for this model, so never send a thinking-disabled setting, and never force a tool choice; both are rejected. Effort defaults to medium; set it per route in output_config. max_tokens covers the thinking AND the answer together, so give chat at least 2000 and retry once with a bigger budget if the text comes back empty.

FINISH PROPERLY: start the server, confirm the page loads and the graph renders, confirm /chat returns a clean placeholder-key error rather than a crash. Then tell me exactly what to paste into config.json and give me three example questions based on my actual note titles.
```

Make it yours: casual chat is happy at low effort and noticeably snappier. Save higher effort for the heavy jobs, like the teardown write-up in Part Two.

PROMPT 02 · SPEECH

The Voice, both directions

Best for: the moment it stops being a website and starts being an assistant.

```
Give it a voice, both directions, using only what the browser already has. No paid speech services.

SPEAKING: every answer is spoken with the Web Speech API. Prefer a British English voice when the system has one. Browsers block audio until the user has interacted with the page, so make the first spoken line fire reliably after my first click, never before.

LISTENING: a microphone button using webkitSpeechRecognition. I click it, I speak, the transcript goes through the same /chat flow as typing.

THE PART EVERYONE GETS WRONG: speech recognition finalizes a phrase when I pause, and I pause in the middle of sentences. Do NOT dispatch on the first final result. Buffer it, with the wait in ONE named constant at the top of the file, FINISH_MS, set to 900. If more speech arrives before the timer fires, append it and restart the timer; the pause that truly ends my thought sends the whole combined sentence. Short interrupt words like "stop" and "wait" must bypass the buffer and fire instantly.

VOICE ROUTES: keep a list of voice routes, each a pattern and a handler, that is checked BEFORE a sentence goes to the brain. Later prompts add their own commands to this list instead of editing the core.

THE MUTE FLAG: if the page URL contains ?mute=1, nothing in that tab ever speaks. I will use that for testing so I never hear the assistant from a background tab. Put the check inside the one function that speaks, so nothing can route around it.

STATUS: a small status line that shows listening, thinking and speaking, so I always know which of us the machine thinks is talking.

VERIFY: give me a test plan: what to click, what to say, and what I should see and hear, including one deliberate mid-sentence pause to prove the finish window works, and one ?mute=1 tab to prove the silence.
```

Make it yours: FINISH_MS is the single most personal number in this build. Too low and it cuts you off mid-thought; too high and it feels sluggish. Start at 900 and tune it after a day of real use, never in one sitting.

PROMPT 03 · THE DEMO MOMENT

Make it prove where the answer came from

Best for: the ten seconds that make people lean toward the screen.

```
When it answers from a note, I want the galaxy to prove it.

/chat already returns a nodes array of the indexes it used. Use it:

- Fly the camera to the top source node, light it and its direct neighbours, and open its side panel while the answer is still being spoken, so I can see exactly which note it came from.
- If the answer drew on four or more notes, do not fly anywhere. Light the whole cluster. Flying to one arbitrary node out of six is a lie about where the answer came from.
- Never read the note aloud unless I ask. It is on screen. The spoken answer stays short.
- Small talk must not drag the camera around. "Good morning" and jokes hold the graph still. Only real questions about my notes move the camera.

That last rule separates a demo from a toy, so implement it deliberately: decide whether the question was about my notes before you decide whether to move the camera.

ONE OWNER FOR FOCUS: only the answer path moves the camera or changes which nodes are lit. Any other feature that wants attention on a note goes through the same functions, never around them.

Then show me: one question that flies to a single note, one that lights a cluster, and one that must not move the camera at all.
```

Make it yours: this is the shot worth recording. Ask something whose answer lives in one very specific note and film the dive. It reads on video better than any feature list.

PROMPT 04 · CHARACTER

The Butler

Best for: turning a question box into something people want to talk to.

```
Give it a personality. Rewrite the /chat system prompt so the assistant is a dry, impeccably polite British butler with a razor wit. Rules:

- Address me as "sir", occasionally, not in every sentence. Over-using it is the difference between charming and grating.
- Answer in ONE witty sentence plus the facts. Never recite the note back; it is on my screen.
- One genuinely funny line beats three bland ones. If nothing funny is available, be brief instead of forcing it.
- When my notes do not cover something, say so plainly and with a little dignity. Never invent a source, never pad, never pretend a related note is the answer.
- Never claim an action happened unless the code actually did it and handed you a receipt. Offering is fine. Saying "done" when nothing happened is the one unforgivable sin.

BOOT GREETING: when the page loads or refreshes it says one short line, "Hello, sir." and nothing more. Show the real indexed note count ON SCREEN, read from the graph data, never hardcoded.

CONTROL TAGS: when I ask for an action the brain cannot do by talking, it replies with a short tag in square brackets instead of prose, and the server executes the tag. Keep the list of tags in ONE place in the persona block; later prompts add their own.

Put the entire persona in ONE clearly commented block at the top of server.py so I can rewrite the character later without hunting through the file.

Then give me five things to say to it that show off the personality, including one question it definitely cannot answer from my notes.
```

Make it yours: the butler is just the default. Swap the persona block for a terse mission-control operator, a sarcastic friend, or a patient teacher. Keeping it in one block is what makes that a thirty-second change.

PROMPT 05 · GROWTH

Total Recall

Best for: the point where the brain grows without you opening an editor.

```
Let me grow the brain by voice.

- When I say or type anything starting with "remember that", POST it to a new /remember endpoint that writes a real
markdown file into a captures/ folder inside my notes directory, with a sensible title from the first few words and
today's date inside the file.
- Add the new node to the running galaxy LIVE, without a page reload: born at the position of its most related existing
node, a brief glow pulse, then fly the camera to it.
- Link it into the graph the same way build.py would, so it is a real citizen of the galaxy and can be a source in the
very next answer.
- Confirm out loud in one line, in character.

ONE DOOR FOR NEW NOTES: put the server side in ONE function (write the file, append the node to graph-data.js, add it
to the live search index, return its index) and the viewer side in ONE function (addNote: birth, link, pulse, fly). A
later prompt births teardown notes through exactly these two.

Two things to get right, because they bite later:
1. Writing a file is not the same as indexing it. The new note must be searchable by /chat in the next question without
re-running build.py.
2. Never let a capture silently fail. If the write fails for any reason, say so out loud. A second brain that quietly
forgets is worse than no second brain.

Then have me test it: I will say "remember that the finish window should be 900 milliseconds", you show me the new star
being born, and I will immediately ask a question that should retrieve it.
```

Make it yours: add trigger phrases that match how you actually talk: "note to self", "don't let me forget", "log this". The capture only works if the phrase is one you would say without thinking.

PROMPT 06 · THE ONE THAT MATTERS

The Preflight

Best for: the difference between a weekend demo and something you rely on.

```
Build me a preflight harness. This is the most important prompt in this pack, so take your time.

Write preflight.py that runs every LIVE chain end to end and prints a verdict. Not unit tests, not mocks. Real calls
against the running system, because the failures that hurt are the ones where every unit test passes and a real chain
is dead.

Check, in order, with a tick or a cross for each:
- The server is up and serving the viewer.
- The graph data loads and the node count is greater than zero.
- /chat returns a well-formed answer to a real question, with a nodes array.
- The API key in config.json is valid, by one real minimal call.
- The model named in config.json is one the key can actually reach: ask the Models API for it by id, and fail with the
id in the message.
- /remember writes a real file and the note is retrievable by /chat immediately afterwards (then remove the test note).
- The files the browser is actually SERVED match the files on disk. A stale served file is the most common cause of "I
fixed it but nothing changed".
- config.json is not reachable from the browser. This one must fail loudly.
- A ?mute=1 tab really is silent: the one speak function refuses when the flag is set.

Finish with one summary line: N pass, N fail, N warn. Exit non-zero on any failure.

Then run it and show me the output. If anything fails, fix the cause and run it again until it is clean. Do not
describe what it would say. Run it.

From now on, whenever you tell me a change is done, run preflight.py first and paste the summary line. "Done" means
preflight passed. Every prompt in Part Two will add checks to this file.
```

Make it yours: add a check every time something breaks on you. A harness that grows one check per real incident becomes the most valuable file in the project within a month. Mine has 60 and every one was earned.

Part Two · the X-ray

Six prompts that give your Jarvis X-ray vision with REA. First the kit, installed safely. Then the clean browser REA watches, the capture itself, the teardown note born in your galaxy, the voice and the panel, and the drill that proves it all before a camera sees it.

The kit	REA installed outside your project, pinned to the version you tested, checked against the official repository.
The browser	A brand-new Chrome profile for every teardown, closed and deleted the moment it is done. Never your own logins.
The capture	REA watches the site run while Jarvis scrolls it, reads its code, and takes a snapshot. Jarvis adds the full file list and the exact versions.
The teardown	Evidence in, a note out: the answer, the stack, how the effect works, a rebuild recipe, and what could not be seen.
The voice	One sentence, out loud, with a live panel that shows every step and a reader for the full note.

Two laws hold across all six prompts. REA observes and never drives: no clicks, no logins, no changes to anything. And every claim cites its evidence: guesses are labelled as guesses, and every test cleans up after itself.

PROMPT 07 · THE KIT

Install REA the Safe Way

Best for: the first paste of Part Two. The tool, pinned and verified, and a bay that never crashes without it.

```
Give my Jarvis a reverse engineering kit: REA (Reverse Engineer Anything, github.com/morluto/rea, MIT licence, npm package rea-agents). Jarvis will use it on WEBSITES ONLY: public sites, open-source code, and my own projects.

INSTALL IT OUTSIDE THE PROJECT: create ~/.jarvis-tools/rea and install there with npm install rea-agents@VERSION --save-exact --ignore-scripts. Before installing, run npm view rea-agents repository.url and confirm it points to github.com/morluto/rea, because copycat packages and repos exist. Use the current version, then PIN it: never install "latest" at runtime.

REQUIREMENTS: REA needs Node.js 22.19+ (or 24.11+). Find node on the PATH or in the common install folders, check its version, and call REA as node HOME/node_modules/rea-agents/scripts/rea.mjs COMMAND --json, so it still works when the server starts without my shell's PATH. Find Google Chrome on Mac and Windows.

THE BAY: teardown.py, Python standard library only, soft-imported by server.py so Jarvis boots fine without it. GET /teardown/diag returns: ready true or false, the installed and pinned REA versions, the Node version, whether Chrome was found, and a plain-English list of problems. When anything is missing, a teardown request answers in one line, in character, with what to install, and nothing crashes.

THE LAW: REA observes. It never clicks, types, logs in, or changes anything, and Jarvis never points it at anything but websites I may look at.

VERIFY: show me /teardown/diag reporting ready, the pinned version, and the exact install commands you ran.
```

Make it yours: REA changes fast. When a new version ships, upgrade on purpose: bump the pin, rerun the drill in Prompt 12, and only then use it on camera.

PROMPT 08 · THE BROWSER

A Clean Browser, Every Time

Best for: giving REA something to watch, without ever touching your own Chrome.

```
REA never opens a browser itself; it attaches to one we own. Build that browser.

CLEAN EVERY TIME: for each teardown, create a brand-new profile folder under ~/.jarvis-tools/rea/profiles/JOB_ID and launch Chrome with --user-data-dir set to that folder, --remote-debugging-port=9333 (loopback only), --no-first-run, --no-default-browser-check, --disable-extensions, a 1280 by 820 window, and the site's address. Never my everyday Chrome, never my cookies or logins. A fresh profile also keeps file sizes honest: a reused one serves files from its cache and makes a heavy site look tiny.

WAIT PROPERLY: poll http://127.0.0.1:9333/json/version until it answers (25 seconds at most), find the page target for the site's host (allow www and redirects), then wait until document.readyState is complete, plus 2.5 seconds for the intro animation.

A SMALL DEVTOOLS CLIENT: write a minimal standard-library WebSocket client for the page's webSocketDebuggerUrl (handshake, masked client frames, large and fragmented replies, ping and pong). It has two jobs only: Runtime.evaluate for read-only page facts, and Input.dispatchMouseEvent mouseWheel to scroll. Use wheel events, not scrollTo, because smooth-scroll libraries listen to the wheel.

ALWAYS CLOSE: in a finally block, terminate Chrome, kill anything still using that profile path, and delete the folder. A teardown must never leave a second Chrome running.

HEADLESS FOR TESTS: a flag that runs the same browser with no window, so a test never pops up on my screen.

VERIFY: open a public page headless, print its title and the seconds until it finished loading, scroll it, close it, and prove that no Chrome is left running with that profile and the folder is gone.
```

Make it yours: on camera, leave the window visible. A browser that opens, scrolls a famous site by itself and closes again is the best b-roll in this whole build.

PROMPT 09 · THE CAPTURE

Watch It Run, Read the Code

Best for: the evidence. Three REA commands plus the two things REA deliberately doesn't do.

```
Now the evidence, captured from the clean browser.

1. WATCH WHILE SCROLLING: get the target id with rea list-browser-targets, then start rea inspect-web-page
http://127.0.0.1:9333 TARGET_ID --observation-ms 7000 --include-script-sources in a background thread. One second
later, scroll the page with about 16 wheel steps, so scroll effects and lazy files fire while REA is watching.
2. READ THE CODE: rea analyze-web-bundle on the same target: which file loads which, the routes, and the strings the
code hands to the graphics card (a name like nearCoCBuffer means depth of field).
3. SNAPSHOT: scroll back to the top with wheel events, then rea capture-web-screenshot. Decode the PNG and save it.
4. THE ASSET LOG (Jarvis, not REA): REA only reports the site's own origin, and only from the moment it attached. So
also read performance.getEntriesByType('resource') through the DevTools client: every file, every origin, with sizes.
Group them by kind (3D models, compressed textures, video, audio, fonts, WebAssembly, scripts) and keep the 12 largest.
5. FINGERPRINTS (Jarvis): search the code REA captured for license banners, which survive minification: "Three.js
Authors" followed by the revision number, "GSAP 3.x.y", "postprocessing v6.x". Keep the patterns in ONE table, ordered
by what matters to a designer: the 3D engine, animation, smooth scroll, visual effects, then compression and build
tools. Report any other banners separately.

Label every line of evidence with who captured it, REA or Jarvis's own browser reads. The brain will be told to keep
that attribution straight.

VERIFY: run all five against a public 3D website headless and paste a short digest: scripts with sizes, routes, the top
code signals, the stack with versions, and the asset totals.
```

Make it yours: the fingerprint table is where your taste lives. Add the libraries your audience cares about, and order them so the panel always leads with the ones people ask about.

PROMPT 10 · THE TEARDOWN

Evidence In, a Note Out

Best for: turning raw evidence into something you can read, hear, and keep.

```
Turn the evidence into a teardown I can read, hear, and keep.

THE DIGEST: compress everything into plain text under about 15,000 characters: the site, my question, page facts,
scripts, which file loads which, routes, code signals, the stack with versions, asset totals and the largest files,
third-party origins, and the limits (REA attaches to a running page; library names come from banners and strings).

THE BRAIN: one call to my current brain. System prompt: every claim must trace to the evidence; say "likely" for
inferences; never invent a library, file or technique; keep the REA versus Jarvis attribution straight. Output EXACTLY:
LABEL: Teardown · HOST, then SPOKEN: two or three sentences for my butler to say (the headline answer first, the most
surprising detail, no web addresses or file names), then SUMMARY: one short paragraph, then ---NOTE--- and a markdown
note with: a one-line verdict, The answer, The stack (each item with its evidence), How the effect works (numbered),
What it loads, Rebuild it (5 to 7 steps for Claude Code), What the evidence can't show. Give it at least 7000 max
tokens: thinking and answer share the budget.

THE NOTE: write it to NOTES/captures/teardowns/HOST-DATE.md with the snapshot saved in an assets folder and embedded,
plus an Evidence section with the stack, the totals and REA's evidence ids. Index it with the same function Total
Recall uses, under an [[REA]] concept note that is created the first time and linked to the Jarvis note. Birth both
through the same addNote: REA first if it's new, then the teardown, with the camera flying to it. Speak the SPOKEN
line.

ONE AT A TIME: a second request while one is running gets "I'm already taking one apart, sir."

VERIFY: run a full teardown headless, then show me the note, the spoken line, and the new nodes. Reread the note
against the digest and flag any claim the evidence doesn't support.
```

Make it yours: the "What the evidence can't show" section is the most important one. It is what makes your teardowns trustworthy enough to show a client.

PROMPT 11 · THE VOICE

One Sentence, Out Loud

Best for: the moment it becomes a feature people want to try.

```
Make it one sentence, out loud, with something to watch.

THE ROUTES (voice routes, checked before the brain): catch ONLY sentences that name a site: "reverse engineer igloo dot inc", "tear down linear.app", "how does stripe.com do that", "what is example.com built with", plus "this site" and "this page". Turn speech into an address: "dot" becomes ".", "slash" becomes "/", "colon" becomes ":", localhost allowed. On a Mac, "this site" reads the address of my front Chrome tab with osascript, never the Jarvis tab itself; elsewhere, ask me for the name. "Tear down the old shed" and "how should I reverse engineer my pricing" must NOT trigger anything.

THE TAG: for sentences with no address ("reverse engineer Stripe's homepage"), the brain replies [TEARDOWN:SITE|QUESTION] and nothing else, using the word this for the current tab. The server starts the job and answers with a short acknowledgment.

THE PANEL: a compact card, top-left, labelled with the site and a small "via REA" chip, showing 8 plain steps live: Opening the site in a clean browser, Waiting for it to load, REA is watching while I scroll, REA is reading the code, Taking a snapshot, Listing everything it loaded, Identifying the stack, Writing the teardown. When it's done it folds to one line ("Taken apart in 45 s"), then the snapshot, up to 10 stack chips, the totals, and a "Read the full teardown" button that opens the whole note in a reader with Open in Obsidian and Close. It never auto-hides, never covers my answer or other panels (measure them and shrink to fit, dropping the snapshot first), and never shows file paths on screen.

MORE ROUTES: "show me the last teardown" (the newest note on disk after a restart), "open the full teardown", "close the teardown", "cancel the teardown".

VERIFY: in a ?mute=1 tab, run 12 sentences through the routes and show which trigger, then run one teardown and screenshot the panel at 1440 by 900.
```

Make it yours: "how does this site do that?" is the line to film. You're on the site, you ask the question you actually have, and Jarvis goes and finds out.

PROMPT 12 · THE PROOF

The Teardown Drill

Best for: proving the whole kit before a camera ever sees it, without leaving a trace.

```
Prove the teardown kit before a camera ever sees it.

OFFLINE CHECKS (fast, no browser): five spoken addresses parse correctly ("igloo dot inc", "stripe.com slash pricing",
"localhost:5001", "make it pop" gives nothing, and "wikipedia.org" keeps its first letter), a short synthetic code
string fingerprints as Three.js r165 and GSAP 3.12.5, and every evidence line in the digest names its source.

A LIVE DRILL THAT LEAVES NO TRACE: snapshot the graph data file, the episode log and the teardowns folder; run one
headless teardown of a public page; check the panel steps, the note sections, the spoken line, the node births and the
time; then restore the snapshot and delete the test note, so my first real teardown on camera is genuinely the first.

SAFETY CHECKS: the endpoints that serve the note and its images refuse any path outside the teardowns folder (try
../config.json and expect a refusal), no investigation Chrome is left running after a run or a crash, and the profile
folders are gone.

PREFLIGHT GROWS THREE CHECKS: the kit is ready with the pinned REA version, the offline checks pass, and no
investigation Chrome is running.

VERIFY: run the drill and preflight, paste both summary lines, then give me a four-step script for my first on-camera
teardown: the site, the exact sentence, what I should see, and what I should hear.
```

Make it yours: run the drill again every time you change the fingerprints, the panel or the REA version. A teardown that quietly breaks only shows up on camera.

Seven things I learned giving Jarvis X-ray vision

1 **REA watches. It never drives.**
It attaches to a browser you own and only observes: no clicks, no typing, no navigation. So Jarvis owns the browser, opens it, scrolls it and closes it, and REA does the looking.

2 **On a 3D site, the page itself is almost empty.**
igloo.inc is one screen tall with 27 page elements. Everything that matters lives in a 1.4 MB code file. Reading the page tells you almost nothing; you have to watch it run.

3 **REA arrives after the party starts.**
On my first test it reported zero network requests, because it attached after the page had loaded. The browser's own log had all 95 files. Now Jarvis reads both.

4 **License banners are a cheat code.**
"Three.js Authors" followed by a revision number survives minification, so Jarvis names exact versions instead of guessing: Three.js r165, GSAP 3.12.5, postprocessing 6.35.5.

5 **A reused browser lies about weight.**
With a reused profile, igloo.inc weighed 0.4 MB, because everything came from the cache. With a fresh one, 14.7 MB. A brand-new profile, every run.

6 **Label the guesses, and say who saw what.**
The best detail in the igloo teardown, a 1.2 MB texture named "scroll data", is a guess from a file name, and the note says so. The brain also once credited REA with something Jarvis read itself; now every evidence line names its source.

7 **Test without leaving fingerprints.**
Every test teardown writes a real note and real nodes into your brain. Snapshot before, restore after, so your first teardown on camera is the real first.

> **The receipt.** igloo.inc, taken apart in 45 seconds from one spoken sentence: 95 files, 14.7 MB, Three.js r165, GSAP 3.12.5, depth of field and bloom, and a page that is one screen tall because the "scroll" moves a camera through a 3D world.

Where to go from here

Run Part One in an afternoon, live with the butler for a day, then run Part Two one prompt at a time and run the drill before you trust it on camera. The first time Jarvis takes apart a site you have admired for months and tells you how it really works, you will never look at a website the same way.

1 **Post your best teardown.**
The free community has a thread for every pack. A screenshot of your panel and the one surprising thing Jarvis found is the best post you can make.

2 **Rebuild one.**
Hand the "Rebuild it" section of a teardown to Claude Code and build your own version, with your own assets and your own twist. Learning how the best sites work is the fastest way to build like them.

3 **Turn it into a service.**
With an owner's permission, a teardown of their website shows exactly what is slowing it down: the heaviest files, missing compression, and the pile of third-party scripts. That is an audit businesses will pay for.

Want the finished Jarvis without building it?

Inside the paid AI Workshop community you get the full Jarvis build I run every day, with a five-minute install and no code, plus the classroom, the live calls, and a team that helps when something breaks.

Free community: skool.com/aiworkshop-lite
Full workshop: skool.com/aiworkshop
YouTube: Zubair Trabzada | AI Workshop